

An algorithm to fix timing by useful skew

B12901137 楊祐宇 B12901113 林偉宸 B12901037 曹丞豐

Abstract

We propose a two-phased algorithm to solve the problem D of ICCAD 2026 CAD Contest. Our algorithm first generates a better clock tree structure by converting the problem into an LP problem without objective function, replacing and inserting buffers according to the result, then, we use an iterative greedy method to modify the clock tree until it does not improve. Our method has improved the score a lot on all official testcases.

1. Introduction

The problem of Problem D is as follows: A clock tree and several paths between two flip-flops are given, each with its SS delay and FF delay. It asks to minimize the score by inserting new buffers or replacing existing buffers with buffers inside the given buffer library.

The skew of each path can be computed with following formula:

$$D_{clk}^{launch} = \sum_k d_{buf,k}^{launch}, D_{clk}^{capture} = \sum_k d_{buf,k}^{capture}$$

$$\Rightarrow Skew = D_{clk}^{capture} - D_{clk}^{launch}$$

where $d_{buf,j}^{launch}$ and $d_{buf,k}^{capture}$ are the launch buffer delay and capture buffer delay under the timing path respectively. The setup slack and the hold slack of each path is computed as follows:

$$Slack_{setup}^{(i)} = T_{clk} - T_{setup} - D_{data}^{(i)} + Skew$$

$$Slack_{hold}^{(i)} = D_{data}^{(i)} - T_{hold} - Skew$$

In this problem, the value of T_{setup} and T_{hold} are 0.08 T_{clk} and 0.05 T_{clk} respectively.

The score is defined as follows:

$$Score = \alpha \left(1 - \frac{TNS_{SS}}{TNS_{SS}^{opt}} \right) + \left(1 - \frac{WNS_{SS}}{WNS_{SS}^{opt}} \right) + \beta \left(1 - \frac{TNS_{FF}}{TNS_{FF}^{opt}} \right) + \left(1 - \frac{WNS_{FF}}{WNS_{FF}^{opt}} \right) + \gamma \left(1 - \frac{Area}{Area_{opt}} \right)$$

where TNS and WNS are total negative slack and worst negative slack, whose maximum value is clamped to zero. Each buffer is given its Fast-Fast (FF) corner, and Slow-Slow (SS) corner, representing the fastest and slowest operating condition. The hold time slack will be computed with FF corner, and the setup time will be computed with SS corner.

The value of α , β and γ are not provided. However, it is stated that α (corresponding to setup slack) is the most important. Hence, we may focus on minimizing the negative setup slack first. We assume that $\alpha = 0.6$, $\beta = 0.2$, $\gamma = 0.2$ in this essay.

2. Methodology

In this section, we will introduce the component and the workflow of our algorithm.

2.1 Overview

The algorithm we proposed consists of two phases. In the first phase, we convert the problem into a LP problem without objective function, generating a solution for FF and SS delays of each buffer to minimize the biggest negative slack among every path. Then, we replace and insert buffers so that the FF and SS delays of each buffer fit our previous calculation

the most. In the second phase, we perform a 2-round iterative greedy algorithm, greedily replacing buffers that maximize the score in each iteration, choosing a different initial structure in each round. The total algorithm is time-limited to 585 seconds, reserving 15 seconds to output answers.

2.2 First phase

The first phase aims to find a better clock tree structure quickly. This phase does not take area into consideration.

Initially, we convert the setup-time constraints of each path into a multi-objective linear programming problem, using an MOLP-solver to solve the problem. While this method works well on problems with small scales, it has little-to-no use on those with larger scales, which is basically every testcase after the testcase change on June 15th.

Hence, we modify the algorithm. We still convert the problem into an LP problem, but the structure of the problem is changed, mainly without an objective function. Each buffer and flip-flop corresponds to a variable, representing the difference between their new total delay and their current total delay, and the constraints are constructed in the following way:

1. For each buffer and its child, we construct:

$$lo \leq orig + (buf) - (child) \leq hi$$

where lo and hi are the minimum and maximum delay of the library buffers with the same fanout number as the buffer we are examining, (x) is the variable corresponding to x , and $orig$ is the current delay of the buffer.

This ensures that the calculated delay of each buffer does not exceed the SS/FF delay range of the buffers with the same fanout number in the library.

It should be noted that total delay here is different from delay. A “delay” is the signal delay between the fanin and the fanout of a buffer, and a “total delay” is the signal delay between a flip-flop and the root of the clock tree. For a buffer or a flip-flop, its total delay is the summation of the delays of its ancestors, similar to D_{clk} .

2. For each path from launching flip-flop to capturing flip-flop, we construct:

$$base + (capture) - (launch) \geq \tau$$

where $base$ is the original slack of the path, and τ is a predetermined parameter. This ensures that the slack of every path is higher than τ .

Note that the corresponding SS and FF constraints in the cases listed above are taken into consideration separately. They form independent inequalities and do not affect each other.

Unlike the previous attempt, we do not utilize a LP-solver to solve the problem. Instead, we take inspiration from the work of other students (葉又銘 et al., 2026), transforming the linear problem into a graph.

For every variable in LP problem, it is converted into a node; For every constraint $x \leq B - A \leq y$, it is transformed into two directed edges (See Figure 1). By running Bellman-Ford with the root of the clock tree as source and some calculation, we may work out the

Figure 3. The score of our algorithm.
The left value is from the original clock tree
and the right value is from our algorithm.

	SS TNS	FF TNS	Area	Score
testcase 0	-1134.2/-121	-2.98/-0.05	6997/3691	1.20
testcase 1	-252.4/-31.2	-0.41/-0.02	1653/953	1.12
testcase 2	-98.7/-7.4	-0.10/-0.00	980/566	1.38
testcase 3	-374.2/-44.0	-1.02/-0.05	2784/1627	1.13
testcase 4	-2277.7/-239.4	-6.99/-0.07	10539/5411	1.28

—X

Figure 1. The graph for the example constraint

required delay for each buffer, or finding out that the answer does not exist if a negative cycle exists. This ensures that the answer will be returned once it is found without care of objective function, unlike using a LP solver.

As for τ , we use binary search to determine the maximum τ . we set the *low* variable as the current negative slack, and finding *high* by iterating the abovementioned algorithm, with each round increasing τ exponentially, until a negative cycle occurs. Then, we use binary search between *high* and *low* to find the maximum τ producing a feasible solution.

The abovementioned algorithm will run twice. The first run focuses on existing buffers. After the first run, the paths with remaining negative slacks are not able to be fixed just by resizing, hence demanding more “freedom” and needing to insert new

buffers. We mark the parents of the launching and capturing flip-flops of the violating paths. A new buffer will then be inserted under the marked buffer. Then, the second run will perform.

We introduce “zero buffer”, which is an imaginary buffer having zero delay and area. Only new inserted buffers can be replaced by the zero buffer in the second run of phase 1 or in phase 2.

Finally, each buffer will be replaced with a buffer whose SS delay is the closest to the calculated delay, generating the final solution for the first phase.

2.3 Second phase

The second phase proceeds with the solution from the first phase, aiming to find a better solution. In this phase, we greedily replace buffers to maximize the score.

In this phase, a 2-round greedy algorithm will be performed. The algorithm shown in Figure 2. will be iterated constantly in each round.

In the first pass, *priority* = *weight*. The *weight* is calculated before the start of each round as follows: for each path with negative slack, every weight of the node who is either the ancestor of the launching flip-flop or that of the capture flip-flop (which is, every buffer that can change the slack of the path) will add ($\max(0, -\text{setup_slack}) + \max(0, -\text{hold_slack})$). This ensures that the buffer that harms the slack the most will be adjusted first. The weight will not change in the same round.

In the following passes, the order of buffers will be shuffled by an algorithm similar to the reservoir algorithm (Efraimidis & Pavlos, 2006) in the following way: each buffer is assigned a random

Figure 2. A pass of Greedy Algorithm

Algorithm 1 A pass of Greedy Algorithm
Sort each buffer by <i>priority</i>
<i>improved</i> $\leftarrow$ <i>True</i>
while Not timeout or <i>improved</i> do
<i>improved</i> $\leftarrow$ <i>False</i>
for Each buffer <i>b</i> , iterate in order do
$b_R, \text{score} \leftarrow 0, 0$
for Each buffer <i>lb</i> in the library that are not <i>b</i>
and with the same fanout number as <i>b</i> do
Calculate the score <i>ls</i> if replacing <i>b</i> with <i>lb</i>
if <i>ls</i> > <i>score</i> then
$b_R, \text{score} \leftarrow lb, ls$
end if
end for
if <i>score</i> > original score then
Replace <i>b</i> with b_R
original score $\leftarrow$ <i>score</i>
<i>improved</i> $\leftarrow$ <i>True</i>
end if
end for

number u between 0~1 in each pass, and $u^{1/\text{weight}}$ will be the next *priority* of the buffer. This method ensures diversity in the heuristic.

The iteration will continue until 50 continuous passes without improvement or timeout occur, thus marking the end of the round.

In the first round, the initial clock tree will be the clock tree structure from the first phase; In the second round, the initial clock tree will be the original clock tree structure. This prevents the first phase from limiting the improvement of the clock tree.

Between two clock trees generated in two rounds, the one with the best score will be chosen. After that, every imaginary buffer, the newly inserted buffer having zero delay, will be removed, thus generating the modified clock tree structure.

3. Evaluation

Our algorithm is evaluated using the new testcases given in the ICCAD contest, which is updated on June 15.

As shown in Figure 3, our algorithm effectively increases SS TNS and FF TNS and halves area, obtaining a good score (the score is calculated with $\alpha = 0.6$, $\beta = 0.2$ and $\gamma = 0.2$).

As for Figure 4, It can be seen that phase I halved both the SS TNS and FF TNS, while slightly increasing area. This

Figure 4. The score of our algorithm after Phase I.

The left value is from the original clock tree and the right value is from our algorithm.

	SS TNS	FF TNS	Area	Score
testcase 0	-1134.2/-596	-2.98/-0.59	6997/7836	0.60
testcase 1	-252.4/-133.1	-0.41/-0.23	1653/1907	0.47
testcase 2	-98.7/-42.2	-0.10/-0.04	980/1065	0.88
testcase 3	-374.2/-190.4	-1.02/-0.68	2784/3178	0.51
testcase 4	-2277.7/-1155.0	-6.99/-3.65	10539/11285	0.70

exchange is acceptable, considering the fact that the runtime of this phase is usually lower than 10 seconds. Comparing it to the first phase of the previous algorithm, it is far better than the previous method (MOLP), as shown in Figure 5.

It should be noted that, due to our algorithm consisting of nondeterministic steps, the algorithm's results may change slightly.

4. Conclusion

We propose a two-phase algorithm to Problem D of ICCAD Contest 2026. We first convert the problem into an LP problem without an objective function and solve it to generate a better clock tree structure. Then, we use a 2-round iterative greedy algorithm to maximize the score. Our method can greatly reduce SS TNS and FF TNS, also halving Area.

5. References

- 2026積體電路電腦輔助設計軟體製作競賽 (June 18 2026) ICCAD Contest.
https://www.iccad-contest.org/tw/03_problems.html
- Efraimidis, Pavlos S.; Spirakis, Paul G. (2006-03-16). "Weighted random sampling with a reservoir". *Information Processing Letters*. 97 (5): 181–185.
[doi:10.1016/j.ipl.2005.11.003](https://doi.org/10.1016/j.ipl.2005.11.003).

Figure 5. The score of the Phase I of the previous algorithm.

Testcase	0	1	2	3	4
SS WNS	-0.53/ - 0.53	-0.40/ - 0.40	-0.30/ - 0.30	-0.46/ - 0.46	-0.59/ - 0.59
SS TNS	-1134/ - 1133	-252/ - 251	-98.7/ - 98.6	-374/ - 373	-2278/ - 2273
FF WNS	-0.11/ - 0.11	-0.08/ - 0.08	-0.03/ - 0.03	-0.11/ - 0.11	-0.12/ - 0.12
FF TNS	-2.99/ - 2.99	-0.41/ - 0.41	-0.10/ - 0.10	-1.02/ - 1.02	-6.99/ - 6.99
Area	6997/6908	1653/1630	980/966	2784/2744	10539/10408
Score	0.003059	0.006733	-0.000119	0.004298	0.003735

This algorithm also takes inspiration from other teams who integrates Bellman-Ford Algorithm into their algorithm, like the team of 葉又銘、張睿恆 and 郭彥聖.

6. Appendix

Github Link. The current version of code is in the branch “submission”.

[ExuberantIris/TestCADContestD: A test repo for 2026 CAD Contest D](#)

About AI. Our code is written in assistance with AI agents. We come up with the sketch of the algorithm, exploiting the AI agent to complete the detail. Also, we use the AI agent to organize the workflow of the algorithm to help write this essay. Besides that, this essay and the slide are written completely by humans.

Contribution.

楊祐宇	建構演算法主架構與Code撰寫、部分 Essay 撰寫
林偉宸	部分程式優化和構思、檢查slide與essay內容
曹丞豐	提供早期的Code, 負責Slides與Essay的撰寫